

CELEBAL TECHNOLOGIES

WEEK 8 ASSIGNMENT — INTERNSHIP MINI PROJECT REPORT

E-Commerce Order Analytics System

End-to-End Data Engineering & SQL Analytics Pipeline



Name	Harshita
CEI ID	CT_CSI_DE_199
Domain	Data Engineer
Batch	2027
University	MMDU
Organization	Celebal Technologies
Assignment	Week 8 Assignment – Intern Mini Project
Project Duration	3 - 4 Weeks
Tools & Technologies	Python (pandas, numpy), MySQL Workbench 8.0, mysqlconnector-python
Report Date	13 July 2026

Executive Summary

This report documents the design, development, and execution of the E-Commerce Order Analytics System, completed as the Week 8 intern mini-project. The objective of the project was to simulate a real-world business intelligence workflow: generating messy, realistic e-commerce data, cleaning and validating it programmatically, loading it into a relational database, and extracting actionable business insights using SQL - including window functions, common table expressions (CTEs), and cohort-style analysis.

The project was executed in five structured phases - synthetic data generation, data cleaning and validation, SQL analytics, Python-SQL integration for automated reporting, and edge-case testing. Four interconnected datasets (customers, products, orders, and order_items) totaling roughly 2,000 records were generated with deliberately embedded data-quality issues (missing values, malformed dates, inconsistent casing, and referential violations) to mirror the kind of "messy" data encountered in production systems.

All intentional defects were successfully detected and resolved through custom Python cleaning functions, and the cleaned data was loaded into a MySQL database with proper primary/foreign key constraints. Sixteen SQL queries of increasing complexity were then written to answer real business questions - from category-wise revenue to customer cohort retention - followed by a command-line Python tool that queries the database directly and produces on-demand summary reports.

Table of Contents

1. Project Background & Objectives.
2. System Architecture & Workflow.
3. Relational Data Model.
4. Phase 1: Synthetic Data Generation.
5. Phase 2: Data Cleaning & Validation.
6. Phase 3: SQL Analysis (16 Queries).
7. Phase 4: Python + SQL Integration.
8. Phase 5: Edge Case Handling.
9. Challenges & Key Learnings.
10. Conclusion.

1. Project Background & Objectives

Modern e-commerce businesses process thousands of orders daily, and the raw data generated across systems is rarely clean. Missing customer identifiers, inconsistent date formats, invalid emails, and orphaned records are common. This project simulates joining a company in exactly that situation: the task was to build, end-to-end, a pipeline that ingests messy raw data, cleans and validates it, stores it relationally, and produces trustworthy business reports.

1.1 Objectives

- Generate four realistic, interlinked datasets with intentionally embedded dataquality issues.
- Build reusable Python functions to detect and clean those issues (nulls, bad formats, casing, referential integrity).
- Design and populate a normalized relational database (MySQL) with primary and foreign key constraints.
- Write SQL queries spanning basic aggregation to advanced window functions, CTEs, and cohort analysis.
- Build a command-line Python tool that connects to the database and generates ondemand summary reports.
- Validate system robustness through a structured set of edge-case tests.

1.2 Skills Demonstrated

- Python data manipulation with pandas and numpy
- Data quality auditing and cleaning strategy
- Relational database design (primary keys, foreign keys, normalization)
- Advanced SQL: window functions, CTEs, subqueries, self-joins
- Python-to-database integration using mysql-connector-python
- Structured software testing / edge-case validation

2. System Architecture & Workflow

The project follows a linear, five-phase pipeline architecture. Raw data is synthetically generated first, then cleaned and validated in Python, loaded into a relational MySQL database, analyzed using SQL, and finally exposed through a Python command-line reporting tool. Each phase produces a well-defined output that becomes the input of the next, mirroring a typical production ETL (Extract-Transform-Load) and analytics workflow.

This staged approach was chosen deliberately: it isolates concerns (generation vs. cleaning vs. analysis vs. reporting), makes each stage independently testable, and reflects how real data teams structure ETL pipelines feeding into BI/reporting layers.

3. Relational Data Model

The database schema consists of four tables connected through primary and foreign key relationships, forming a typical star-like order-management model: a customers table and a products table act as dimension tables, while orders and order_items capture transactional (fact) data.

3.1 Schema Design Rationale

- customers.customer_id is the primary key referenced by orders.customer_id, enabling per-customer aggregation.
- products.product_id is the primary key referenced by order_items.product_id, enabling category/product-level rollups.
- orders.order_id is the primary key referenced by order_items.order_id, linking each line item back to its parent order.
- order_items stores quantity, unit_price, and discount_percent at the line-item level, so revenue is always derivable rather than pre-aggregated - this keeps the model normalized and audit-friendly.

4. Phase 1: Synthetic Data Generation

The first phase generated four interlinked CSV datasets using Python (pandas, numpy, and the random module), with a fixed random seed for reproducibility. Rather than producing clean data, the generation logic deliberately injected realistic data-quality defects, so that the subsequent cleaning phase would have genuine issues to detect and resolve - closely mimicking how raw operational data actually looks.

4.1 Datasets Generated

Dataset	Rows	Key Columns
customers.csv	500	customer_id, customer_name, email, registration_date, customer_type
products.csv	500	product_id, product_name, category, subcategory, cost_price
orders.csv	500	order_id, customer_id, order_date, status, region_code
order_items.csv	500	item_id, order_id, product_id, quantity, unit_price, discount_percent

Referential integrity between orders and order_items was preserved by default (each order_items.order_id was sampled from existing orders.order_id values), with a small, controlled set of exceptions intentionally introduced later to test integrity-checking logic (see table below).

4.2 Intentional Data-Quality Issues Introduced

Issue	Affected File	Volume	Description
Invalid emails	customers.csv	10 rows (2%)	Emails replaced with a malformed string (e.g. missing @/domain)
Missing emails	customers.csv	10 rows (2%)	Email set to NaN / blank
Mixed-case product names	products.csv	20 rows (4%)	Product name forced to UPPERCASE
Extra whitespace	products.csv	20 rows (4%)	Leading/trailing spaces added around product name
NULL customer_id	orders.csv	25 rows (5%)	customer_id set to NaN to simulate missing linkage

Wrong date format	orders.csv	15 rows	order_date rewritten as DD-MM-
-------------------	------------	---------	--------------------------------

Issue	Affected File	Volume	Description
		(3%)	YYYY instead of the standard format
Negative quantity	order_items.csv	15 rows (3%)	Quantity set negative to simulate returns
Orphaned order_id	order_items.csv	10 rows (2%)	order_id set to a non-existent value (O9999)
Discount over 100%	order_items.csv	5 rows (1%)	discount_percent set to 120 (invalid range)

Each dataset was saved in both CSV and Excel formats and archived into raw_data/ and reports/ working folders to keep raw and processed outputs clearly separated, and to preserve an auditable trail from raw input to cleaned output.

5. Phase 2: Data Cleaning & Validation

With the raw datasets in hand, Phase 2 focused on systematically auditing and resolving every category of defect introduced in Phase 1. The process began with a full null-value and duplicate-record audit across all four tables, followed by a series of purpose-built cleaning functions, each targeting a specific data-quality problem.

5.1 Cleaning Functions Implemented

Function	Purpose
<code>clean_customers()</code>	Strips whitespace from <code>customer_name</code> , lower-cases email, fills missing emails with a placeholder value
<code>validate_emails()</code>	Flags any row whose email does not contain "@"; used to quantify and later fix invalid emails
<code>clean_products()</code>	Trims leading/trailing whitespace and converts <code>product_name</code> to Title Case for consistency
<code>clean_orders()</code>	Fills missing <code>customer_id</code> with "Unknown" and normalizes <code>order_date</code> to a single consistent datetime format using pandas' mixed-format parser
<code>clean_order_items()</code>	Converts negative quantities to their absolute value and caps <code>discount_percent</code> at 100
<code>check_referential_integrity()</code>	Identifies <code>order_items</code> rows whose <code>order_id</code> does not exist in the <code>orders</code> table, then removes them

5.2 Referential Integrity Enforcement

The `check_referential_integrity()` function specifically searched `order_items` for rows referencing an `order_id` absent from the `orders` table. This surfaced the 10 intentionally orphaned records (`order_id` = "O9999") created in Phase 1. These rows were then filtered out of `order_items`, and the check was re-run to confirm zero remaining violations before the data was considered clean.

5.3 Output

The cleaning phase produced fully validated CSV and Excel exports (`customers_clean`, `products_clean`, `orders_clean`, `order_items_clean`) ready for database loading, along with a clear, reproducible record of every issue detected and how it was resolved.

6. Phase 3: SQL Analysis

With clean data loaded into the ecommerce_order_analytics MySQL database (customers, products, orders, order_items, with foreign key constraints enforced), sixteen SQL queries were written and executed in MySQL Workbench 8.0, organized into three tiers of increasing complexity: basic aggregation, intermediate business-rule logic, and advanced analytics using window functions, CTEs, and self-joins.

6.1 Basic Queries

#	Query	What It Does
1	Total Revenue per Category	Aggregates quantity × unit_price × (1 - discount%) across order_items, grouped by product category, to identify the highest-revenue categories.
2	Top 10 Customers by Order Value	Ranks customers by total order value using a JOIN and GROUP BY, returning the ten highest-spending customers.
3	Month-wise Order Count	Counts orders grouped by YEAR() and MONTH() of order_date to reveal ordering trends across the year.

6.2 Intermediate Queries

#	Query	What It Does
4	Customers with No Delivered Orders	Uses a NOT IN subquery to find customers whose orders never reached DELIVERED status - a churn/fulfilment risk signal.
5	Products with More Returns than Purchases	Uses conditional SUM (CASE WHEN) to compare RETURNED vs. non-returned order counts per product, surfacing problem SKUs.
6	Return Rate per Category	Calculates (returned items / total items) × 100 per category using conditional aggregation, rounded to two decimals.

6.3 Advanced Queries (Window Functions, CTEs, Subqueries)

#	Query	What It Does
7	Running Totals (Window Function)	SUM() OVER (PARTITION BY region_code ORDER BY order_date) computes a cumulative revenue total per region over time.
8	Ranking with	Ranks products by revenue within each category using
#	Query	What It Does

	DENSE_RANK	DENSE_RANK() OVER (PARTITION BY category ...), ensuring tied revenues share a rank.
9	LAG/LEAD Analysis	LAG(order_date) OVER (PARTITION BY customer_id ...) combined with DATEDIFF() computes the gap in days between each customer's consecutive orders.
10	Multi-Level CTE	A CTE first computes monthly revenue per customer, then a CASE expression buckets each into High/Medium/Low tiers, aggregated by month.
11	NTILE Segmentation	NTILE(4) OVER (ORDER BY total_value DESC) splits customers into four value-based quartiles labeled Platinum/Gold/Silver/Bronze.
12	Year-over-Year Comparison	Groups revenue by YEAR() and category to allow direct comparison of category performance across years.
13	First/Last Value Analysis	FIRST_VALUE() and LAST_VALUE() (with a full frame clause) return each customer's first and most recent order date side by side.
14	Cumulative Distribution	CUME_DIST() OVER (ORDER BY total_value DESC) shows what percentile of total customer value each customer falls into.
15	Cohort Analysis (Complex CTE)	Groups customers by registration month (cohort) and LEFT JOINS to orders to measure how many customers per cohort placed at least one order.
16	Self-Join with Window Function	A CTE computes per-order revenue, then LAG() OVER (PARTITION BY customer_id ORDER BY order_date) compares each order's value to the customer's previous order.

These sixteen queries collectively cover the full range of SQL analytics techniques typically expected in a data-analyst or data-engineering role: multi-table joins, conditional aggregation, ranking functions, running totals, cohort/retention analysis, and selfreferencing window comparisons - all validated against the cleaned dataset.

7. Phase 4: Python + SQL Integration

Phase 4 connected the SQL analysis layer to an interactive Python command-line tool, using the mysql-connector-python library, so that business users could generate ad-hoc summary reports without writing SQL themselves.

7.1 Tool Workflow

- Connects to the ecommerce_order_analytics MySQL database using mysql.connector, with error handling around the connection attempt.
- Prompts the user for a report type (Daily/Weekly/Monthly) and a start and end date.
- Runs a parameterized summary query returning total orders, total revenue, and unique customers within the selected date range.
- Runs a second query returning the top 3 best-selling products (by quantity) within that range.
- Runs a third query computing total revenue before the start date, then calculates the percentage change versus the current period - safely handling the case where no prior data exists (division-by-zero guard).
- Closes the database cursor and connection cleanly at the end of execution.

7.2 Sample Output Fields

Metric	Source
Total Orders	COUNT(DISTINCT order_id) within date range
Total Revenue	SUM(quantity × unit_price × (1 - discount%)) within date range
Unique Customers	COUNT(DISTINCT customer_id) within date range
Top 3 Products	Ranked by total quantity sold within date range
Revenue Change %	$((\text{current revenue} - \text{previous revenue}) / \text{previous revenue}) \times 100$

8. Phase 5: Edge Case Handling

To validate the robustness of the pipeline, four automated test functions were written and executed directly against the MySQL database, each targeting a specific data-integrity risk identified in the project brief.

#	Edge Case Tested	Result / Validation Logic
1	order_items references an order_id not present in orders	Query returns zero matching rows after cleaning, confirming referential integrity was fully enforced
2	discount_percent exceeds 100	Query returns zero rows post-cleaning, confirming the 100% discount cap was correctly applied
3	quantity equals 0	Checked explicitly to ensure no zero-quantity line items exist that would distort revenue calculations
4	order_date falls in the future	Checked using <code>order_date > CURDATE()</code> to ensure no invalid future-dated orders exist in the dataset

All four tests executed successfully, and the pipeline confirmed that the cleaning and loading process had eliminated every category of intentionally introduced defect prior to analysis - giving confidence that downstream SQL insights were computed on trustworthy data.

9. Challenges & Key Learnings

- Handling mixed date formats (standard timestamp vs. DD-MM-YYYY) required using pandas' "mixed" format parser with error coercion rather than a single fixed format string.
- Balancing realistic "messiness" with reproducibility - using a fixed random seed ensured the same defects could be regenerated and re-validated consistently across runs.
- Designing conditional aggregation (CASE WHEN inside SUM) for return-rate and returns-vs-purchases queries required careful thought about what "purchase" vs. "return" means in terms of order status.
- Working with window functions like NTILE(), CUME_DIST(), and FIRST_VALUE()/LAST_VALUE() clarified the difference between ranking, distribution, and frame-based window functions in SQL.
- Writing multi-level CTEs (e.g., the cohort analysis and monthly revenue tiering queries) reinforced how to break a complex analytical question into clean, sequential, readable steps.
- Building the CLI reporting tool highlighted the importance of parameterized queries (to prevent SQL injection) and defensive handling of edge cases like a missing "previous period" baseline.

10. Conclusion

This project successfully delivered a complete, working analytics pipeline covering the full lifecycle of business data: from intentionally messy synthetic generation, through systematic Python-based cleaning and validation, into a normalized relational database, and finally into sixteen SQL queries spanning basic reporting to advanced window-function analytics - all exposed through an interactive command-line reporting tool.

Beyond the technical deliverables, the project reinforced core data-engineering and analytics practices: data quality auditing, referential integrity enforcement, normalized schema design, advanced SQL problem-solving, and defensive software testing through structured edge-case validation. These are directly transferable skills applicable to realworld data analyst, data engineer, and BI development roles.